TALLER | 15 MINUTOS

Diagnóstico de arquitectura de SEDA

Un punto de partida para conversar sobre decisiones que ya se ven en el sistema. No es una propuesta de rediseño.

Propósito
Poner nombre a tres decisiones, ubicar el mayor acoplamiento y elegir una preocupación para los próximos sprints.

Tres decisiones que ya están en el código

La clasificación es una hipótesis para iniciar la conversación. “Una vía” significa costosa de revertir, no imposible. El repositorio no contiene registros formales de decisiones (ADR), así que la intención original se debe confirmar con el equipo.

DECISIÓN	PUERTA	¿SE DECIDIÓ A PROPÓSITO?
Supabase como plataforma PostgreSQL, Auth y Storage viven en Supabase. Las cuentas se relacionan con `auth.users` y el servidor crea enlaces temporales para documentos.	**Una vía, en la práctica** Migrar identidad, relaciones y archivos tendría un costo alto.	El stack está declarado en el estándar del proyecto, señal de elección consciente. Falta documentar qué alternativas se evaluaron y por qué.
Una aplicación Express El mismo servidor publica las páginas de `public/` y monta la API.	**Dos vías** Se puede separar más adelante, aunque habrá trabajo de despliegue y comunicación entre partes.	La estructura es simple y consistente con el stack descrito. Confirmar si la decisión de mantener un solo despliegue fue explícita.
Autorización MINED en el servidor El navegador envía un token; el middleware valida identidad, rol y estado antes de algunas rutas.	**Dos vías, costo medio** Se puede cambiar el mecanismo de sesión coordinando cliente y API.	La frontera de autorización sí aparece de forma deliberada en el código. El detalle de guardar el token en `sessionStorage` es más fácil de revisar por separado.

Pregunta para el equipo: ¿Qué parte de cada decisión recuerdan haber acordado y qué parte se fue definiendo sobre la marcha?

DÓNDE SE CONCENTRA EL ACOPLAMIENTO

El caso de equivalencia conecta varias piezas

El ciclo de vida de un caso atraviesa la consulta del panel, el reporte detallado, la decisión del MINED, los documentos y las pruebas. Por eso, un cambio pequeño en los datos o estados del caso puede obligar a revisar varios archivos.

Datos y consultas

`controllers/panel.controller.js` y `controllers/casos.controller.js` consultan relaciones del esquema de Supabase.

Reglas de revisión

`controllers/casos.controller.js` permite revisar solo casos `PENDIENTE` y escribe `ACEPTADO` o `ADVERTENCIA`.

Pantallas

`public/mined.html` presenta la lista; `public/js/reporte-caso.js` presenta el reporte y habilita la revisión.

Documentos y evidencia

`servicios/documentos-caso.js` genera enlaces temporales; las pruebas y la configuración de Hachi describen estados y resultados esperados.

Señal concreta para conversar
La pantalla del panel todavía compara estados en minúsculas como `pendiente` y `aprobado`, mientras el backend usa estados como `PENDIENTE`, `ACEPTADO` y `ADVERTENCIA`. Es un ejemplo de una regla compartida que puede desalinearse entre interfaz y servidor.

¿Los sprints reflejan las fronteras del sistema?

SPRINT	LO QUE SE VE EN EL REPOSITORIO	LECTURA PARA EL TALLER
1	El historial muestra registro de universidades, carga de archivos, registro de equivalencias y panel.	Alcance inferido de los commits; no encontré una fuente de Sprint 1 que confirme el reparto exacto.
2	HU-6: login MINED; HU-8: reporte; HU-9: revisión.	Historias verticales de extremo a extremo, apoyadas en la misma API y entidad de caso.
3	HU-3: cálculo con IA; HU-4: checklist, según la configuración QA.	Extiende el flujo existente de equivalencias. El plan todavía es preparación, no evidencia de implementación.

La cola de cálculo prevista para IA

El plan de Sprint 3 propone una cola en memoria que procesa un caso a la vez. Es una decisión que vale la pena discutir antes de depender de ella en el ambiente compartido.

> **¿Qué puede pasar?**
> Si el proceso se reinicia mientras hay casos esperando, la cola en memoria puede perderlos. Si el despliegue usa más de una instancia, cada instancia tendría su propia cola y habría que evitar que el mismo caso se procese dos veces.

La pregunta no es rediseñarla hoy. Es acordar el límite aceptable: ¿un reinicio puede dejar un cálculo pendiente sin recuperación?, ¿qué pasa si el mismo caso llega dos veces?, ¿el equipo espera una sola instancia en los próximos dos sprints?

También están conectados el proveedor de IA, la extracción de documentos, el tiempo máximo y el costo por caso. El plan QA los trata como dependencias y riesgos por resolver; todavía no acredita su funcionamiento real.

Guion de 15 minutos

TIEMPO	CONVERSACIÓN	RESULTADO
3 min	Nombrar tres decisiones que ya se ven en el sistema.	Lista corta, sin debatir soluciones.
4 min	Clasificar cada una como de una o dos vías; recordar si se acordó o fue evolucionando.	Tres decisiones clasificadas.
4 min	Ubicar el cambio pequeño que hoy toca demasiados archivos y comparar con los sprints 1 y 2.	Acoplamiento principal y una frontera a revisar.
4 min	Elegir qué decisión preocupa más para los dos sprints restantes.	Una preocupación, con dueño o pregunta pendiente.

Para llevarse del taller: las tres decisiones con su clasificación, el punto de mayor acoplamiento y la preocupación seleccionada. Registrar dudas como dudas; no convertir una hipótesis de lectura en una decisión del equipo.

Base consultada: ESTANDAR_CODIGO_SEDA.md; server.js y controladores/rutas actuales; qa/sprints/sprint-2.json y sprint-3.json; tests/PLAN-QA-SPRINT-3.md. Sprint 1 se infiere del historial de commits porque no se encontró su documento de alcance en development.